ADASECANT Optimizer in PyTorch

Checkpoint Presentation: Experiments and Results

Aleksandra Idczak, Igor Lechoszest, Krzysztof Krawiec

May 12, 2026

Agenda

Project Goal

Main objective

Implement the ADASECANT optimizer in PyTorch and evaluate its behavior on several learning tasks.

- Implement ADASECANT as a custom `torch.optim.Optimizer`.
- Compare ADASECANT with Adam on different types of problems.
- Check convergence speed, final loss, accuracy, and training stability.
- Use this checkpoint to evaluate whether the implementation behaves reasonably.

Optimizer Comparison Setup

- The main baseline optimizer is Adam.
- ADASECANT is evaluated using the same model architecture and dataset as Adam in each experiment.
- Each experiment focuses on a different learning scenario:
  - image classification,
  - transfer learning,
  - time series prediction,
  - spatial pattern learning,
  - variational autoencoder training,
  - digit generation.

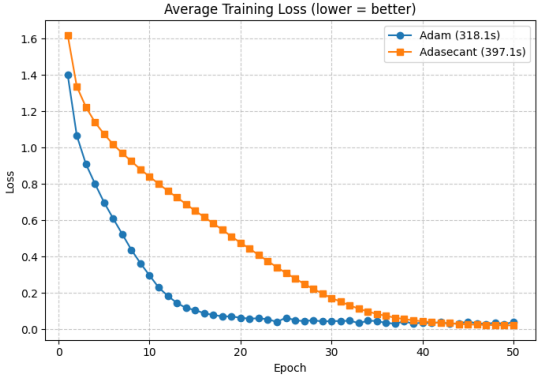
Evaluation criteria

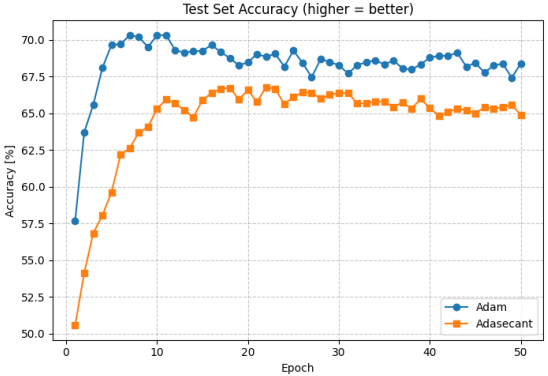
Training loss, test accuracy, MSE, visual quality of predictions, convergence behavior, and training time.

Experiment 1: CNN Image Classification – Methodology

- Task: supervised image classification.
- Model: convolutional neural network.
- Optimizers compared:
  - Adam,
  - ADASECANT.
- Metrics:
  - average training loss,
  - test set accuracy,
  - training time.
- Goal: check whether ADASECANT can train a CNN and how it compares with Adam.

Experiment 1: CNN Image Classification – Results





Experiment 1: CNN Image Classification – Observations

- Adam converges faster in the first epochs.
- ADASECANT reduces training loss more gradually.
- Adam reaches higher test accuracy.
- ADASECANT is slower in this experiment.

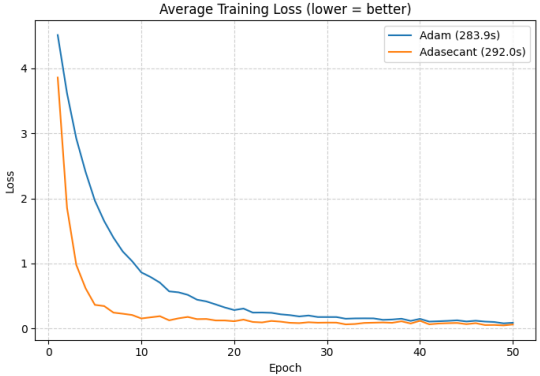
Checkpoint conclusion

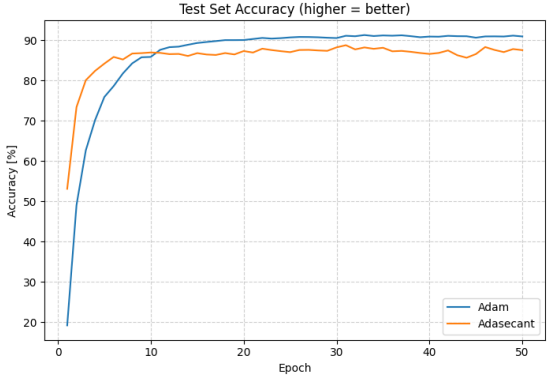
ADASECANT works on the CNN task, but Adam performs better in both accuracy and training time.

Experiment 2: Transfer Learning – Methodology

- Task: image classification using transfer learning.
- A pretrained model is fine-tuned or adapted to the target task.
- Optimizers compared:
  - Adam,
  - ADASECANT.
- Metrics:
  - average training loss,
  - test set accuracy,
  - training time.
- Goal: test optimizer behavior when training starts from pretrained representations.

Experiment 2: Transfer Learning – Results





Experiment 2: Transfer Learning – Observations

- ADASECANT starts with strong early improvement.
- Adam achieves the highest final test accuracy.
- ADASECANT reaches competitive accuracy but remains below Adam.
- Training time is similar, with ADASECANT slightly slower.

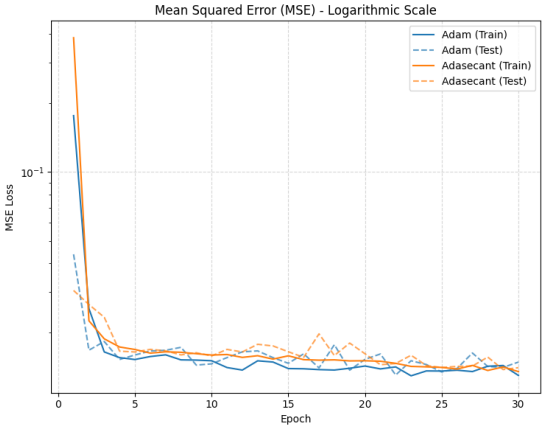
Checkpoint conclusion

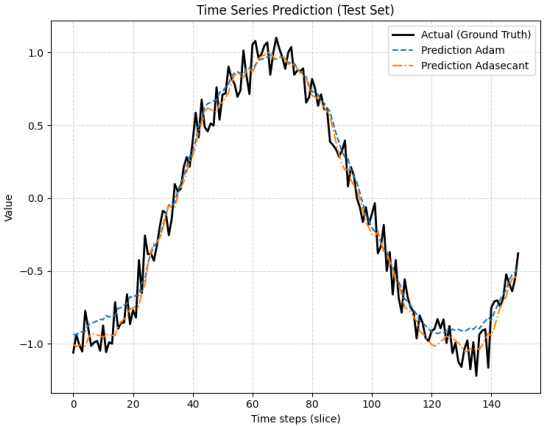
ADASECANT is usable in transfer learning, but Adam remains more effective in final accuracy.

Experiment 3: Time Series Prediction – Methodology

- Task: predict future values of a time series.
- The model is trained to approximate a smooth sinusoidal-like signal.
- Optimizers compared:
  - Adam,
  - ADASECANT.
- Metrics:
  - training MSE,
  - test MSE,
  - visual agreement between prediction and ground truth.

Experiment 3: Time Series Prediction – Results





Experiment 3: Time Series Prediction – Observations

- Both optimizers quickly reduce MSE.
- Adam and ADASECANT obtain very similar predictions.
- The prediction curves are close to the ground truth.
- ADASECANT shows slightly noisier test MSE in some epochs.

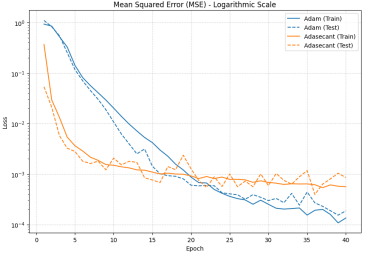
Checkpoint conclusion

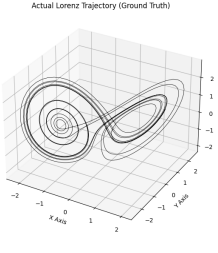
On a simple time series task, ADASECANT performs comparably to Adam.

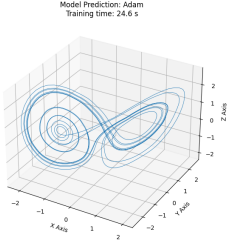
Experiment 4: Spatial Pattern Learning – Methodology

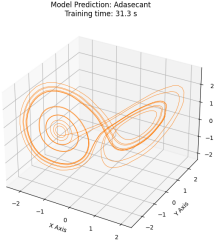
- Task: learn spatial or dynamical patterns.
- The experiment uses a Lorenz-like trajectory as the ground truth.
- Optimizers compared:
  - Adam,
  - ADASECANT.
- Metrics:
  - training loss,
  - test loss,
  - visual similarity of generated trajectory,
  - training time.

Experiment 4: Spatial Pattern Learning – Results









Experiment 4: Spatial Pattern Learning – Observations

- ADASECANT starts with lower loss in the first epochs.
- Adam achieves lower final loss after longer training.
- Both optimizers reconstruct the global trajectory structure.
- ADASECANT is slower than Adam in this experiment.

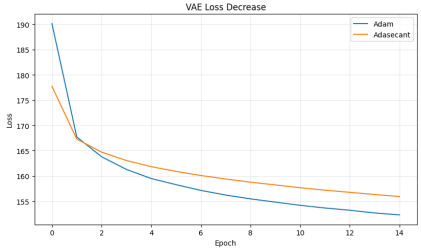
Checkpoint conclusion

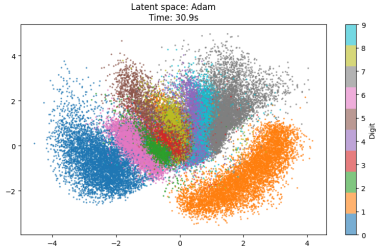
ADASECANT captures the spatial pattern, but Adam reaches better final optimization quality.

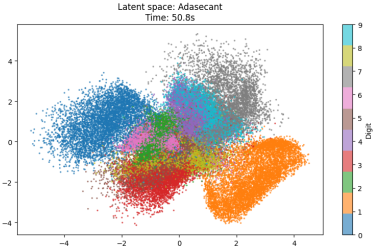
Experiment 5: Variational Autoencoder – Methodology

- Task: train a variational autoencoder on digit data.
- The VAE learns a compressed latent representation.
- Optimizers compared:
  - Adam,
  - ADASECANT.
- Evaluation:
  - VAE loss decrease,
  - latent space visualization,
  - training time,
  - separation of digit classes in latent space.

Experiment 5: VAE Loss and Latent Space







Experiment 5: VAE Observations

- Both optimizers reduce VAE loss.
- Adam reaches slightly lower final loss.
- ADASECANT creates a more separated-looking latent space in this run.
- ADASECANT takes noticeably more time than Adam.

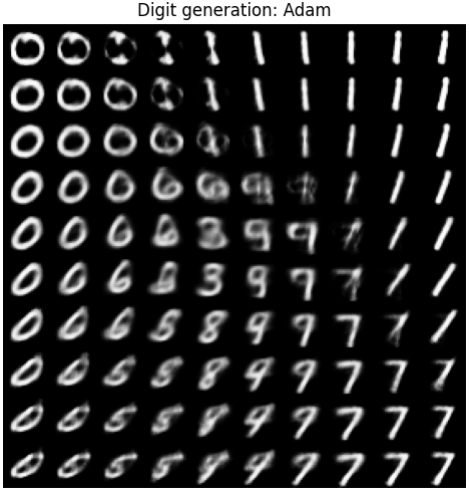
Checkpoint conclusion

ADASECANT can train a VAE, but requires further analysis because visual latent structure and loss do not tell the same full story.

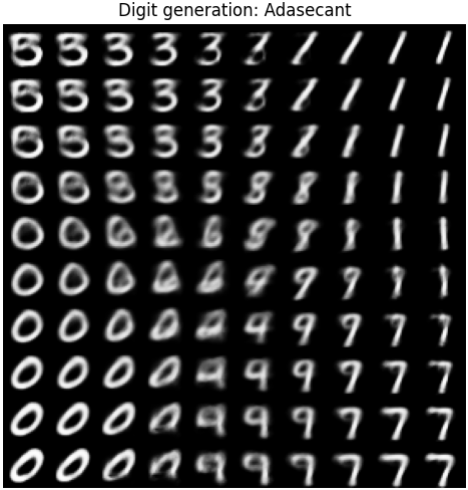
Experiment 6: Digit Generation – Methodology

- Task: generate digits from the trained VAE latent space.
- A regular grid of latent vectors is decoded into images.
- Optimizers compared:
  - Adam-trained VAE,
  - ADASECANT-trained VAE.
- Evaluation is mainly visual:
  - digit sharpness,
  - smoothness of transitions,
  - diversity of generated samples.

Experiment 6: Digit Generation – Adam



Experiment 6: Digit Generation – ADASECANT



Experiment 6: Digit Generation – Observations

- Both optimizers produce recognizable digit-like samples.
- Adam generates smoother transitions between some digit regions.
- ADASECANT generates visible digit structures but some samples appear less diverse.
- More quantitative generative evaluation would be needed for a stronger conclusion.

Checkpoint conclusion

ADASECANT can train a generative model, but the current result should be treated as preliminary.

Overall Results Summary

Experiment	Adam	ADASECANT
CNN classification	Faster convergence, higher accuracy	Stable, but lower accuracy
Transfer learning	Best final accuracy	Competitive but lower final accuracy
Time series prediction	Strong performance	Similar performance
Spatial patterns	Lower final loss	Good reconstruction, slower
VAE training	Lower final loss	Interesting latent separation
Digit generation	Smoother samples	Recognizable but less consistent samples

Main Checkpoint Conclusions

- The ADASECANT implementation works in standard PyTorch training loops.
- The optimizer is able to train classification, regression, dynamical, and generative models.
- Adam is usually faster and often reaches better final metrics.
- ADASECANT shows promising behavior on some tasks, especially simple regression and latent representation learning.
- More experiments are needed before making strong claims about optimizer quality.

Limitations of Current Experiments

- Most experiments are preliminary checkpoint experiments.
- Not all experiments were repeated across multiple random seeds.
- Hyperparameters were not extensively tuned for ADASECANT.
- Runtime is currently worse than Adam in several experiments.
- Some conclusions are based on visual inspection, especially for VAE generation.

Next Steps

- Run experiments with multiple random seeds.
- Add stronger baselines: SGD, RMSprop, AdamW.
- Tune ADASECANT parameters such as `decay` and `gamma_clip`.
- Add tables with final numerical metrics.
- Add unit tests for optimizer state updates.
- Improve documentation of the mathematical update rule.
- Decide which experiments should be included in the final report.

References

C. Gulcehre, M. Moczulski, and Y. Bengio.

ADASECANT: Robust Adaptive Secant Method for Stochastic Gradient.

arXiv:1412.7419, 2014.

C. Gulcehre, J. Sotelo, M. Moczulski, and Y. Bengio.

A Robust Adaptive Stochastic Gradient Method for Deep Learning.

arXiv:1703.00788, 2017.

D. P. Kingma and J. Ba.

Adam: A Method for Stochastic Optimization.

arXiv:1412.6980, 2014.

Thank you